RSCL — Algorithm 10: MA'AT Plan Dry-Run Check (/api/plan/check, _PlanScratch)

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/festival/control_server_hu.py`	`sha256 3f32578a144c`	2026-11-11 06:32 UTC	clean (HEAD 983eede; last change 10ee643)

1. Purpose and context

Higher-level planners (the mission coordinator, Algorithm 11) need to know, before anything moves, whether a sequence of commands would be accepted by the flight supervisor and what separations it would plan. Re-implementing the supervisor's checks in the planner would risk divergence. MA'AT plan check instead executes the **same** `command()` / `formation()` / `_plan_stagger()` / `_plan_group_move()` code on a detached copy of the fleet whose publisher is disabled. The first step is therefore validated exactly as a real command would be at that instant; later steps are validated on a predicted state.

2. Inputs, outputs, data structures

- **Input:** `steps`: non-empty list (≤ 12) of `{action, targets, params}`, action ∈ {formation, turn, spin, move, takeoff, land, hover}.
- **_PlanScratch**: subclass of Fleet with deep-copied Drone objects, private lock, no Gazebo node, no event log, `fresh()` frozen to the set of aircraft fresh at the snapshot instant, and `publish()` that raises.
- **Output:** `{ok, read_only: true, checked_utc, snapshot_monotonic, telemetry (per aircraft x,y,z,v,mode,fault,fresh,age_s,target), steps: [...]}`; each step: `index, predicted, ok, checked, reason, planned_targets, planned_goals, pairwise_min_sep, planned_min_sep`, optional `staggered_wait_for, skipped_stale, formation_offset_m, shape`.

3. Algorithm

Algorithm 10.1 PLAN-CHECK

Source: festival/control_server_hu.py: Fleet.plan_check, _PlanScratch.describe, _PlanScratch.settle

```
function PLAN-CHECK(steps)
  require steps is a non-empty list with |steps| ≤ 12
  with fleet lock: t ← now; F ← { n : fresh(n) }; snap ← deepcopy(drones)      // atomic snapshot
  scratch ← _PlanScratch(snap, F);  failed ← false
  for i, step in enumerate(steps) do                                          // outside the live lock
    entry ← {index: i, predicted: i > 0}
    if failed then entry ← not checked ("an earlier step was rejected"); continue
    validate shape of step (action ∈ PLAN_ACTIONS, targets ⊆ NAMES non-empty, params object)
    before ← positions and modes in scratch
    try
      res ← scratch.command(action, targets, params)                          // unchanged supervisor code
      entry ← ok + DESCRIBE(before, res);  SETTLE(scratch)
    except ValueError, TypeError, OverflowError, KeyError as e: entry ← not ok, reason e
    if not entry.ok then failed ← true
  return {ok: not failed, read_only: true, telemetry, steps}
function DESCRIBE(before, res)
  goal_n ← (x, y, queue_alt) if n QUEUED else (target_x, target_y, target_z) for n ∈ res.targets
  start_n ← before_n;  end_n ← goal_n if n targeted else start_n
```

```
  for each pair (a, b) of the five aircraft do
    if both are parked (z < 0.30) at start and end then skip
    best ← ∞
    for phase in (A, B), for k = 0..40, f = k/40 do
      g(n) ← (f if phase = B else 0) for queued n; (f if phase = A else 1) otherwise   // A: movers; B: queued climb
      best ← min(best, ‖lerp(start_a, end_a, g(a)) − lerp(start_b, end_b, g(b))‖)
    pairwise[a|b] ← best
  return goals, pairwise, planned_min_sep = min(pairwise)
procedure SETTLE(scratch)                                                  // predicted completion
  QUEUED → at (x, y, queue_alt), HOVER;  TAKEOFF/MOVE/TURN/SPIN/FORMATION → at setpoint, yaw = target yaw,
  mode HOVER (FORMATION stays FORMATION);  LANDING → z = 0.05, GROUNDED;  all velocities ← 0
```

4. Data-flow diagram and flowchart

No state machine. `plan_check` keeps no state between calls: each call takes its own snapshot, builds a scratch copy, and discards it on return. The only mode mapping it applies is the deterministic SETTLE prediction (QUEUED → HOVER at queue altitude; TAKEOFF, MOVE, TURN, SPIN → HOVER at the setpoint; FORMATION stays FORMATION at the setpoint; LANDING → GROUNDED at z = 0.05 m; all velocities zero). The data-flow diagram therefore shows what is read and what can never be written.

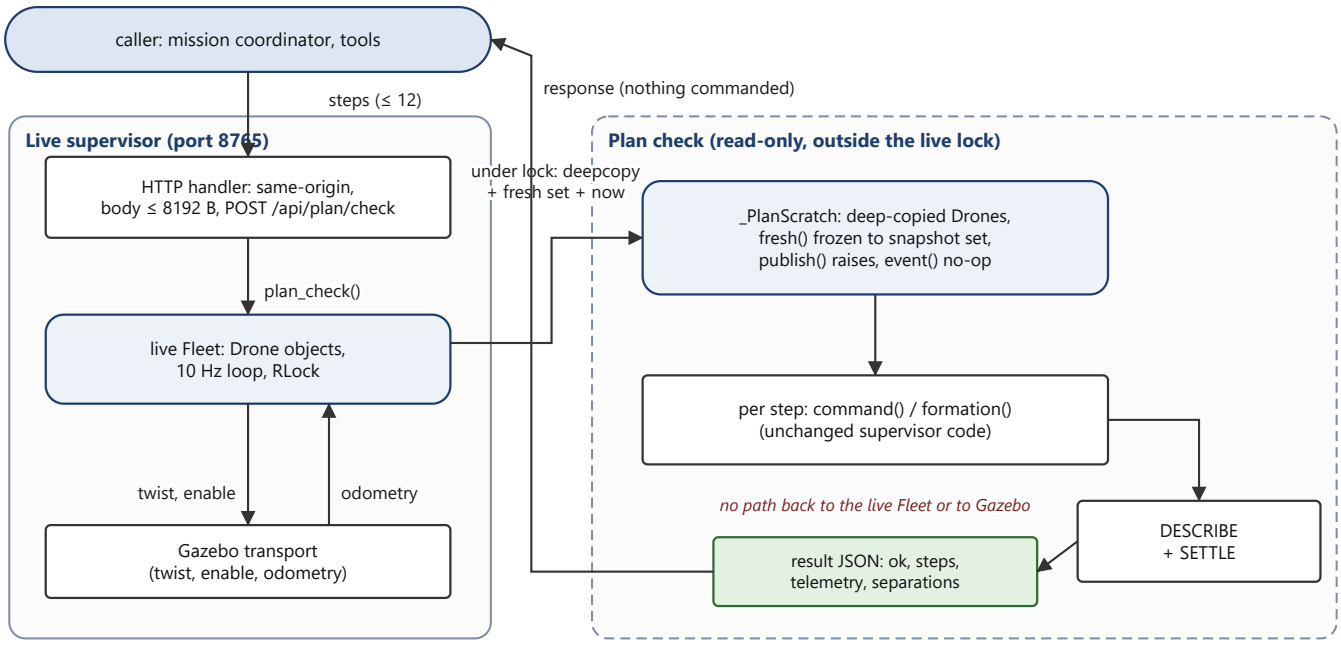


Figure 1. Data flow of a MA'AT plan check. The live fleet lock is held only while copying; the scratch object has no Gazebo node and a `publish()` that raises, so there is no path from the check to the aircraft.

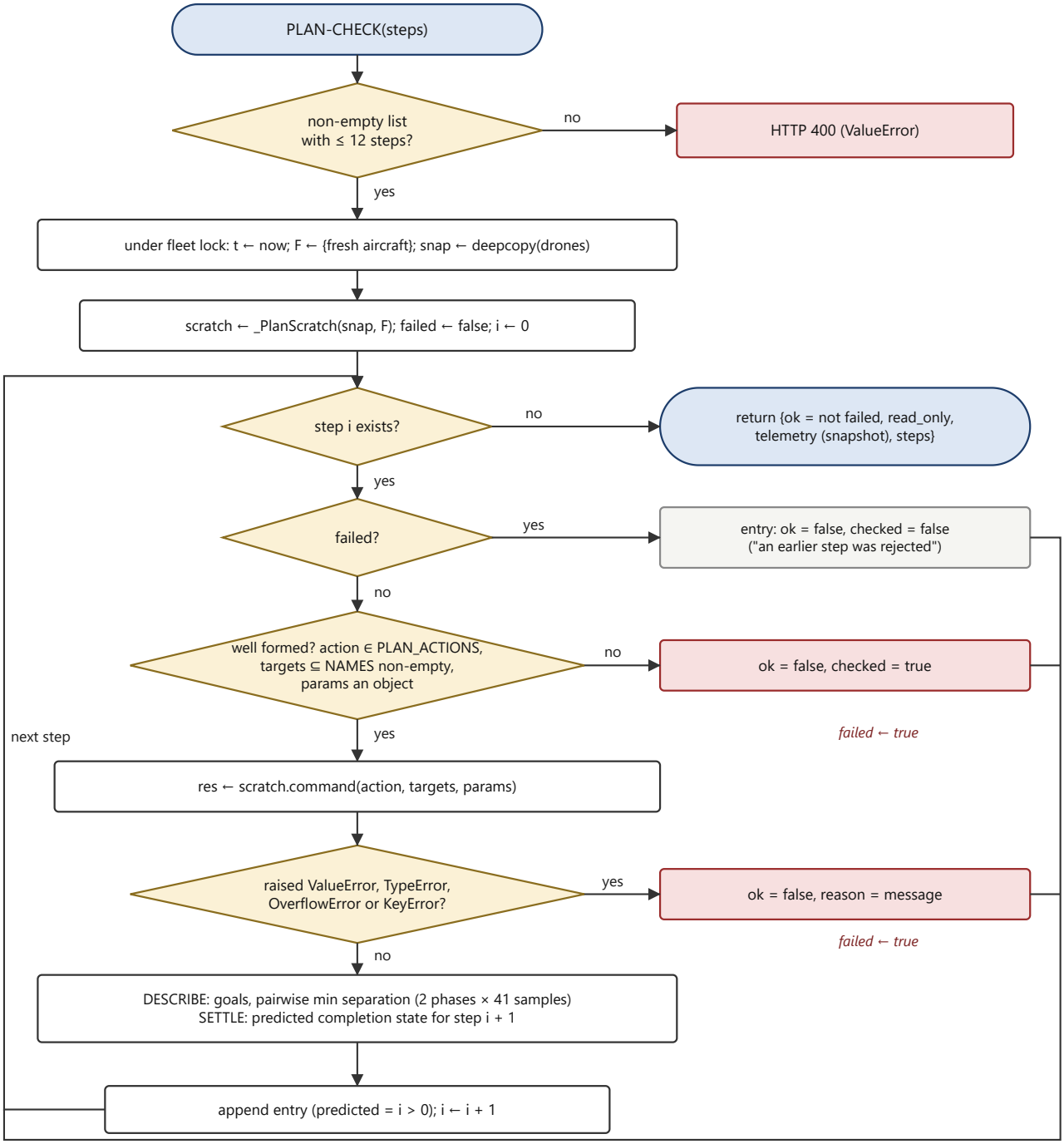


Figure 2. Flowchart of `Fleet.plan_check`. Step 0 runs on the exact snapshot; every later step runs on the state predicted by SETTLE. After the first rejection, later steps are reported but not checked.

5. Parameters

Parameter	Value
Max steps	12
Separation samples	41 per phase, two phases (A movers, B queued climbers)
Parked threshold for omission	z < 0.30 m at both ends for both aircraft
Freshness	frozen at the snapshot (live rule: odometry < 2.0 s)
HTTP	`POST /api/plan/check`; same-origin check; body ≤ 8192 bytes; no token required (read-only)

6. Complexity

Per step: the cost of the command itself (dominated by formation assignment, Algorithm 5: up to ~$8 \cdot 10^5$ distance evaluations with a shifted formation) plus DESCRIBE: C(5,2) = 10 pairs × 82 samples. Deep copy O(N). The live lock is held only for the snapshot.

7. Properties

Guaranteed

- **Read-only:** the scratch object owns copies; any attempt to publish raises; no event is logged; the live state is never mutated.
- **Exactness of step 0:** it is validated by the identical code path and the identical state (positions, modes, faults, freshness) a real command would see at the snapshot instant, including refusal messages.
- **Fail-fast:** after the first rejected step, later steps are reported "not checked" rather than evaluated on an invalid state.
- The 10 Hz control loop is not stalled by the (possibly slow) permutation search.

Not guaranteed

- Steps after the first are validated on an idealised settled state (every setpoint reached exactly, zero velocity); real arrival errors and timing differ.
- Freshness is frozen: an aircraft fresh at the snapshot is assumed fresh for all predicted steps.
- The separation report is a straight-line, synchronised, sampled prediction (two phases for staggered take-off); it does not model the controller (Algorithm 9) and is informational: acceptance depends only on the supervisor's own checks.
- Between the dry run and a real dispatch the world may change; callers must re-check immediately before dispatch (Algorithm 11 does).

8. Testing and measured results

- `acceptance/tests/test_mission_coordinator.py::TestPlanCheck`: `test_plan_check_never_mutates_live_state`, `test_first_step_identical_to_command_path`, `test_rejection_reason_matches_command_message`, `test_fault_and_stale_rejected_without_mutation`, `test_later_steps_predicted_and_skipped_after_failure`, `test_unsupported_actions_refused`, `test_scratch_cannot_publish`; `TestHTTP`: `test_plan_check_endpoint_read_only`, `test_plan_check_cross_origin_rejected`, `test_plan_check_bad_body`.
- Part of the offline suite run by `acceptance/run_offline.sh` (44 tests OK in `run_20261111T052914Z`). Accuracy of predicted separations versus flown separations is not measured for this endpoint. A related data point is the 2026-11-12 live catalogue run, whose test computes the planned minimum separation with the same model DESCRIBE uses (synchronised straight lines from the start positions to the supervisor's assigned slots): for the 44 VALIDATED transitions the observed minimum differed from plan by −0.18 m (`v_right`: 1.279 planned, 1.098 observed) to +0.10 m (`big_cross`); for `pentomino_f` → `pentomino_w`, with proximity holds, by −0.25 m (0.884 planned, 0.633 observed).

9. Comparison with alternatives and related work

Checking a plan before execution and monitoring it during execution are established topics: plan validators check a plan against a formal domain model [1]; runtime verification checks execution traces against formal properties [2], including on robot middleware [3]; verified runtime monitors carry a proof from a hybrid-system model to the running system [4]; runtime-assurance architectures switch to a verified safe controller when a monitor predicts a violation [5], [6]. MA'AT plan check is a dry run of the real implementation, not a check against a model.

Approach	Mechanism	What it guarantees (under its assumptions)	Compared with ours
Plan validation, VAL [1]	Simulate the plan on the semantics of a PDDL domain model, including continuous effects.	The plan is valid with respect to the model.	Ours validates against the executable supervisor code itself, so step 0 has no model-implementation gap; but there is no declarative model and later steps use an idealised settle.
Runtime verification [2], ROSRV [3]	Monitors synthesised from formal properties observe the running system (for ROSRV: ROS messages) and flag or block violations.	Sound detection of violations of the specified property on the observed trace.	Ours checks before dispatch; the runtime monitors in HU-MA'AT (watchdog, coordinator VERIFY) are hand-written, not synthesised from a specification.
Verified runtime validation, ModelPlex [4]	Monitors derived by proof from a verified hybrid-system model; passing the monitor implies the model's safety proof applies.	Safety proof transfers to the execution as long as monitors pass.	Ours has no proof at any level; separations are sampled straight-line predictions.
Runtime assurance: Simplex [6], SOTER [5]	An unverified advanced controller runs while a decision module switches to a verified safe controller when needed.	Safety guaranteed by the safe controller and the switching rule.	Ours has a fallback (hold, then emergency hover) but neither the fallback nor the switching condition is verified.
HU-MA'AT plan check (this document)	Unchanged supervisor command path on a deep copy; predicted settle between steps.	Read-only; step 0 identical to a real command at the snapshot instant (Section 7).	—

Where ours is weaker. No formal semantics, no proof, no coverage of timing or controller behaviour, frozen freshness, and predicted separations that comparable live data show can be optimistic by 0.18 m without and 0.25 m with proximity holds (Section 8). **What it does provide.** Agreement by construction between what is checked and what would be executed for the first step, which a separately written validator cannot provide, at no modelling cost.

10. Relation to other algorithms

Wraps Algorithms 5–9 (command path) without modification; is the validation authority ("MA'AT") for Algorithm 11 and is replayed offline by `OfflineSupervisor` in the coordinator.

11. References

1. R. Howey, D. Long and M. Fox, "VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL," *Proc. 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 294–301, 2004, doi:10.1109/ICTAI.2004.120. `https://nms.kcl.ac.uk/planning/software/val.html`
2. M. Leucker and C. Schallhart, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009, doi:10.1016/j.jlap.2008.08.004. `https://doi.org/10.1016/j.jlap.2008.08.004`
3. J. Huang, C. Erdogan, Y. Zhang, B. Moore, Q. Luo, A. Sundaresan and G. Rosu, "ROSRV: Runtime verification for robots," *Runtime Verification (RV 2014), LNCS 8734*, pp. 247–254, 2014, doi:10.1007/978-3-319-11164-3_20. `https://doi.org/10.1007/978-3-319-11164-3_20`
4. S. Mitsch and A. Platzer, "ModelPlex: Verified runtime validation of verified cyber-physical system models," *Formal Methods in System Design*, vol. 49, no. 1, pp. 33–74, 2016, doi:10.1007/s10703-016-0241-z. `https://doi.org/10.1007/s10703-016-0241-z`
5. A. Desai, S. Ghosh, S. A. Seshia, N. Shankar and A. Tiwari, "SOTER: A runtime assurance framework for programming safe robotics systems," *Proc. 49th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 138–150, 2019, doi:10.1109/DSN.2019.00027. `https://arxiv.org/abs/1808.07921`
6. L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001, doi:10.1109/MS.2001.936213. `https://doi.org/10.1109/MS.2001.936213`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.